Foundations

Senior Data Scientist roadmap

B0 Start here: foundations for data science

B0-L1 Python essentials for data work

Put logic in typed functions that return results, use comprehensions to filter, and test for missing values with `is None`.

```python
def mean_weight(weights: list[float]) -> float | None:
    """Average weight, or None for an empty list."""
    if not weights:
        return None
    return sum(weights) / len(weights)
```

Test for a missing number	if value is None. Truthiness also treats 0 and 0.0 as false.
Filter a list in one line	A comprehension: [w for w in weights if w > 0].
Show 0.125 as 12.5%	f"{share:.1%}".
Walk through a dict	for key, value in d.items():
Signal bad input	raise ValueError("message naming the input"). Catch it with try/except where you can handle it.

B0-L2 pandas essentials

Filter with bracketed masks, summarise with groupby and named aggregations, and merge with how="left" when every row must survive.

```python
import pandas as pd

df = pd.read_csv("respondents.csv")
```

```python
df.groupby("region", as_index=False).agg(
    n=("w", "size"),        # rows, missing or not
    recorded=("w", "count"),  # non-missing values
    total=("w", "sum"),
)
```

Filter rows on two conditions	df[(df["w"] > 0) & df["in_scope"]]. Each comparison in brackets, & not and.
Count rows against count values	size counts rows, count counts non-missing values. sum and mean skip missing values.
Several results per group	df.groupby("region", as_index=False).agg(n=("w", "size"), total=("w", "sum")).
Join and keep every row	df.merge(other, on="key", how="left"). The default inner merge drops rows with no match.

B0-L3 The statistics you need for this track

Report an estimate with its standard error or interval, know which SD your library computes, and read a p-value for what it is.

Standard error of a mean and of a share	Mean: SD / sqrt(n). Share: sqrt(p(1 - p) / n). Four times the sample halves the SE.
A rough 95% interval	Estimate plus or minus 1.96 standard errors.

What a p-value is	The probability of a difference at least this large if there were no true difference. Not the probability that a hypothesis is true.
np.std against pandas std	np.std divides by n (population). Series.std divides by n - 1 (sample). Use np.std(x, ddof=1) for the sample SD.

B1 Project setup

B1-L1 One pyproject.toml and a src layout

Logic lives once, in an installable package. Notebooks, jobs and tests import it.

```
reach_model/
├── pyproject.toml
├── src/reach_model/        # importable package
│   ├── __init__.py
│   ├── features.py
│   └── train.py
├── tests/                  # pytest, imports reach_model
├── notebooks/              # exploration; imports reach_model, holds no logic
└── data/                   # small samples only; real data stays out of Git
```

What goes in pyproject.toml?	[project] with dependencies, [dependency-groups] for development tools, and [tool.*] settings for ruff, mypy and pytest.
Install a dependency group	pip install --group dev (pip 25.1 and later).
Why the src layout?	Only the installed package is importable, so tests run against it exactly as jobs do.
Where does shared logic live?	In the package under src/, imported by notebooks, jobs and tests. Never copied.

B1-L2 One virtual environment per project, and python -m pip

One environment per project, created from pyproject.toml, with python -m pip so installing and running use the same interpreter.

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -e . --group dev
```

Create and use a project environment	python -m venv .venv, activate it, then python -m pip install -e . --group dev.
Why python -m pip?	It installs into the interpreter that python runs. Bare pip may belong to another Python.
Detect a virtual environment in code	sys.prefix != sys.base_prefix.
Two dependencies that cannot coexist	Two pinned environments and two dependency groups, each tested in CI.

B1-L3 ruff and mypy in daily work

ruff and mypy found an unused import, a silent truncation and a crash on zero weight, before anyone wrote a test.

```
m.py:1:1: I001 [*] Import block is un-sorted or un-formatted
m.py:1:8: F401 [*] `os` imported but unused
m.py:9:30: B905 `zip()` without an explicit `strict=` parameter
```

```
m.py:14: error: Unsupported operand types for * ("None" and "int")  [operator]
m.py:17: error: Function is missing a type annotation  [no-untyped-def]
```

Why does ruff flag zip() without strict= (B905)?	zip silently truncates to the shorter input. strict=True raises ValueError instead.
mypy says None * int is unsupported. The fix?	Handle the None case in code. Never # type: ignore it away.
Daily commands before a commit	ruff check --fix ., ruff format ., mypy.
Missing stubs for one library	Install types-<lib> if it exists, otherwise a per-module mypy override. Never global.

B2 Code design

B2-L1 Pure functions, side effects at the edges

The pure core is tested with three rows and a fixed year. The edge is a few lines that are easy to read.

```python
def weighted_mean(values: list[float], weights: list[float]) -> float:
    return sum(v * w for v, w in zip(values, weights, strict=True)) / sum(weights)
```

```
read (impure) -> transform, compute (pure) -> write (impure)
```

What makes a function pure?	Its result depends only on its arguments, and it changes nothing outside itself.
Three hidden side effects in data code	Mutating arguments, reading globals, reading the clock or randomness inside the function.
Return a new DataFrame with an extra column	df.assign(col=...). df["col"] = ... changes the caller's frame.
Program shape	Read at the edge, compute in a pure core, write at the edge.

B2-L2 Dataclasses as configuration objects

A frozen dataclass names every setting, rejects unknown ones, and checks values once, when you build it.

```python
from dataclasses import dataclass, field


@dataclass(frozen=True)
class ReportConfig:
    weight_col: str
    min_base: int = 50
    waves: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        if self.min_base <= 0:
            raise ValueError(f"min_base must be positive, got {self.min_base}")
```

Why a frozen dataclass for settings?	Named, typed fields, unknown keys rejected, and no assignment after construction.
Change one setting on a frozen config	dataclasses.replace(cfg, min_base=30) returns a new object.
Are dataclass type hints checked at runtime?	No. Validate or convert in `__post_init__`, or use pydantic for external input.

Empty list default on a dataclass field	field(default_factory=list). A plain [] raises ValueError at class definition.

B2-L3 When a class earns its place

Functions by default. A class when state lives across calls, an invariant needs guarding, or several methods share the same data.

```python
class Normaliser:
    def __init__(self, col):
        self.col = col

    def run(self, df):
        return df.assign(**{self.col: df[self.col] / df[self.col].sum()})
```

A class with `__init__` and one run method	A function in disguise. Replace it with a function.
Four reasons a class earns its place	State across calls, an invariant to protect, several operations on the same data, a shared interface.
The deletion test	Turn the methods into functions. If nothing breaks, keep the functions.
Settings with no behaviour	A frozen dataclass.

B3 Clean pandas

B3-L1 Dtypes in pandas 3, nullable and string

Decide dtypes at load time. IDs and codes as str, counts with gaps as Int64.

```python
df = pd.read_csv(
    path,
    dtype={"respondent_id": "str", "postcode": "str", "household_size": "Int64"},
)
```

Default text dtype in pandas 3.0	str, with NaN for missing values. Only strings are allowed.
int64 vs Int64	int64 is NumPy, no missing values. Int64 is the pandas nullable integer that holds pd.NA.
Keep leading zeros in IDs	Read the column with dtype str in read_csv.
Assign 1.5 into an int64 Series on pandas 3	Raises TypeError instead of silently upcasting.
Where to set dtypes	At load time, from a dtype map in config. Check df.dtypes right after.

B3-L2 Vectorise instead of apply

The strong version is shorter, about 500 times faster on large frames, and handles missing ages on purpose.

Why is df.apply(func, axis=1) slow?	It calls a Python function once per row. np.where measured about 500 times faster on one million rows.
Vectorised if-else in NumPy	np.where(cond, value_if_true, value_if_false). For several conditions, np.select.
What does NaN > 60 return?	False. np.where and np.select send missing values to a real category unless you handle them.

Numeric bands that keep missing values missing	pd.cut(values, bins, labels, right=False).

B3-L3 Method chaining and Copy-on-Write

Under Copy-on-Write, write with .loc in one step, or better, build a new frame with a chain of assign, loc and pipe.

```
df[df["age"] < 18]["weight"] = 0       # df unchanged, ChainedAssignmentError warning
df.loc[df["age"] < 18, "weight"] = 0   # one step: df changed
```

```
result = (
    raw
    .rename(columns=COLUMN_MAP)
    .loc[lambda d: d["age"] >= 18]
    .assign(weight=lambda d: d["weight"].clip(upper=5.0))
    .pipe(add_age_band, bins=BINS, labels=LABELS)
)
```

Copy-on-Write in pandas 3.0	Always on. Indexing results behave as separate objects, copied only on write.
df[mask]["col"] = value on pandas 3	Changes nothing and emits ChainedAssignmentError. Use df.loc[mask, "col"] = value.
Refer to the current step inside a chain	A lambda: assign(x=lambda d: ...) or loc[lambda d: ...].
Call your own function inside a chain	.pipe(func, **kwargs).

B4 Polars and DuckDB

B4-L1 Polars expressions and lazy frames

Scan lazily, filter and select early, aggregate with expressions, and collect once.

```
import polars as pl

shares = (
    df.group_by("wave", "brand")
    .agg(pl.col("weight").sum().alias("brand_weight"))
    .with_columns(total=pl.col("brand_weight").sum().over("wave"))
)
```

Polars eager vs lazy	DataFrame runs each step at once. LazyFrame (scan_*, .lazy()) builds a plan and runs on collect().
Group total on every row in Polars	pl.col("x").sum().over("group").
Polars sum() of an all-null group	0, not null. Report null_count() with totals.
Mean that skips nulls in Polars	pl.col("x").mean(). sum() / pl.len() counts null rows.

B4-L2 DuckDB, SQL over files and DataFrames

One connection passed in, parameters for values, and a None check on aggregates.

```
import duckdb

con = duckdb.connect()   # in-memory database
rel = con.sql("""
    SELECT wave, brand, sum(weight) AS total
```

```
    FROM read_parquet('extracts/*.parquet')
    GROUP BY wave, brand
""")
df = rel.df()    # pandas; .pl() for Polars, .fetchall() for tuples
```

```
con.execute("SELECT count(*) FROM responses WHERE wave = ?", [wave])
```

Query many Parquet files in DuckDB	FROM read_parquet('dir/*.parquet').
Pass a value into DuckDB SQL	con.execute("... WHERE x = ?", [value]). Never an f-string.
Table names from user input	Placeholders cannot bind identifiers. Check the name against an allowlist.
SQL sum over no rows	NULL, returned as None. Check with is None.

B4-L3 Choosing pandas, Polars, DuckDB or Spark

One primary engine. Reduce where the data is read. Convert small results at a clear boundary.

```
lazy_result = pl.scan_parquet("extracts/
  *.parquet").filter(...).group_by(...).agg(...)
summary = lazy_result.collect().to_pandas()     # Polars to pandas for statsmodels
rel = con.sql("SELECT ... FROM read_parquet('extracts/*.parquet')")
top = rel.pl()                                   # DuckDB to Polars
```

Group-by over 10M rows: pandas, Polars, DuckDB	0.22 s, 0.11 s, 0.06 s on 4 cores. All fast enough. Choose by size against memory and where data lives.
Mixing engines	Reduce first in the engine that reads the files, convert the small result last.
pl.from_pandas and the index	Dropped by default. reset_index() first, or include_index=True.
When Spark	Data already in the lakehouse, larger than one machine, or a scheduled production job.

B5 Data validation

B5-L1 DataFrame schemas with pandera

A schema at the point of entry turns a silent bad input into one loud, detailed failure.

```
import pandera.pandas as pa

RESPONSES = pa.DataFrameSchema(
    {
        "respondent_id": pa.Column(str, unique=True),
        "age": pa.Column(int, pa.Check.in_range(16, 99)),
        "weight": pa.Column(float, pa.Check.gt(0)),
    },
    strict=True,
    coerce=True,
)
clean = RESPONSES.validate(raw, lazy=True)
```

pandera Column defaults	nullable=False. Set nullable=True where nulls are expected.
strict=True and coerce=True	strict rejects unknown columns. coerce converts dtypes before checks.
validate(df, lazy=True)	Runs every check and raises SchemaErrors with failure_cases.

SchemaError vs SchemaErrors	Separate classes. SchemaErrors is not a subclass. Catch both, or better, let them raise.

B5-L2 pydantic models for config and input records

Validate external settings and records once, at the boundary, with every rule in the model.

```python
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    weight_col: str
    min_base: int = Field(default=50, gt=0)
    cap: float = Field(default=5.0, gt=1.0)


cfg = WeightingConfig.model_validate(yaml.safe_load(text))
```

pydantic extra keys, default behaviour	Ignored silently. Use ConfigDict(extra="forbid") for config.
pydantic lax mode	Converts "30" and 30.0 to 30, True to 1. Use strict=True to refuse conversions.
model_copy(update=...)	Does not validate. Rebuild with model_validate({**m.model_dump(), ...}).
pydantic or pandera	pydantic for records and settings from outside. pandera for whole DataFrames.

B5-L3 Validate at the boundaries, with severity

Check at boundaries. Collect structured results. Log them all, then stop on CRITICAL.

```python
Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    check: str
    severity: Severity
    passed: bool
    observed: float | None
```

Where to validate	At boundaries: after reading input, before writing output.
CRITICAL vs WARN	CRITICAL stops the run (raise). WARN records and continues.
Order of QA operations	Run every check, log all results, then raise if any CRITICAL failed.
Two coding rules for checks	Guard every division. Test for None with is None, never truthiness.

B6 Reproducibility

B6-L1 Seeds and random generators

Seed once at the edge, pass the generator in, and never use hash() for anything that must be stable.

```python
rng = np.random.default_rng(20260925)      # at the edge, once
```

```
sample = draw_sample(ids, k=500, rng=rng)  # the function receives rng
```

Reproducible randomness in NumPy 2	rng = np.random.default_rng(seed), created once at the edge and passed in.
Why not np.random.seed?	It seeds one global generator that every library shares and advances.
Stable bucket for an ID	int(hashlib.sha256(id.encode()).hexdigest(), 16) % n. Never hash(), which is randomised per process.
Independent streams for workers	rng.spawn(n).

B6-L2 Record the data, config and code behind every result

Hash the input and the loaded config, record the commit and the seed, and store the manifest with the output.

```
manifest = {
    "run_id": run_id,
    "started_utc": datetime.now(timezone.utc).isoformat(),
    "input_sha256": file_sha256(input_path),
    "config_sha256": config_sha256(config),
    "git_commit": git_commit,
    "seed": seed,
    "packages": {"pandas": version("pandas"), "numpy": version("numpy")},
}
```

Three things to record for a rerun	Data (content hash or table version), config (hash of the loaded config), code (Git commit).
Hash a large file	with path.open("rb") as f: hashlib.file_digest(f, "sha256") (Python 3.11+).
Stable config hash	sha256 of json.dumps(config, sort_keys=True).encode("utf-8").
Raw inputs	Immutable: each delivery at a new path, never overwritten.

B6-L3 Pinned dependencies and lock files

Ranges in pyproject.toml, exact versions and hashes in a committed lock file, and --locked in CI.

```
uv lock                  # resolve pyproject.toml ranges, write uv.lock
uv sync --locked         # install exactly uv.lock; fail if it is out of date
uv lock --upgrade-package pandas   # move one package on purpose
```

pyproject.toml vs lock file	pyproject.toml: ranges you support. Lock: exact versions and hashes of everything installed.
uv sync --locked	Installs exactly uv.lock and fails if pyproject.toml changed without a new lock.
Hashes without uv	pip-compile --generate-hashes, then pip install --require-hashes -r requirements.txt.
Why not pip freeze?	It captures one environment's accidents, with no hashes and no reasons.

B7 Exploratory analysis done well

B7-L1 Profile a dataset before you analyse it

Shape and grain, dtypes, missingness, cardinality and ranges, in code, before any analysis.

```python
def profile(df: pd.DataFrame) -> pd.DataFrame:
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing_share": df.isna().mean(),
        "n_unique": df.nunique(),
    })
```

Five profiling questions	Shape and grain, dtypes, missingness, cardinality, ranges.
nunique() and missing values	Excluded by default. dropna=False counts NaN as a value.
Count every row in a duplicate group	df.duplicated(subset=key, keep=False).sum().
Why save the profile?	Compare it with the next delivery to spot source changes early.

B7-L2 Plot before you summarise

Plot distributions, pairs and time series before summarising, with functions that return labelled figures.

Anscombe's quartet	Same mean, variance and correlation (0.816), different shapes. Plot before summarising.
Four first plots	Histograms, scatter plots of modelled pairs, lines over waves, missingness bars.
Testable plotting code	fig, ax = plt.subplots(), set labels on ax, return fig. No plt.show() inside.
Histogram bins	Choose on purpose and try several. Too few hide spikes at codes.

B7-L3 Write assumptions down, then turn them into checks

Log each assumption with evidence and owner, implement its handling per column, and check it on every run.

```yaml
- id: A3
  statement: "score uses 1 to 10; 98 = refused, 99 = don't know"
  evidence: "Questionnaire v4, Q12; profile of waves 1 to 11 shows only 1-10, 98, 99"
  handling: "98 and 99 become missing before any statistic"
  check: "score in 1..10 or missing, after recoding"
  owner: "analysis team"
  last_verified: 2026-09-01
```

```python
MISSING_CODES: dict[str, list[int]] = {"score": [98, 99]}   # from config, per A3


def recode_missing(df: pd.DataFrame, codes: dict[str, list[int]]) -> pd.DataFrame:
    return df.assign(**{col: df[col].mask(df[col].isin(vals)) for col, vals in codes.items()})
```

Fields of an assumptions log entry	id, statement, evidence, handling, check, owner, last_verified.
Recode missing codes safely	Per column: df[col].mask(df[col].isin(codes)). Never a whole-frame replace.

Three kinds of assumption	Structure (grain, keys), coding (codes, scales, units), scope (who is included).
From assumption to code	Handling in config-driven code, and a QA check that runs on every delivery.

B8 Statistical correctness

B8-L1 Survey weights and the effective sample size

Weighted estimates divide by the weight total. Precision uses the effective sample size.

```
weighted_mean = (y * w).sum() / w.sum()      # same as np.average(y, weights=w)
```

```
n_eff = w.sum() ** 2 / (w ** 2).sum()
design_effect = n / n_eff
```

Weighted mean	sum(y * w) / sum(w), over rows with both present. Same as np.average(y, weights=w).
Kish effective sample size	(sum w)^2 / sum(w^2). Design effect = n / n_eff.
Which n for intervals and suppression?	n_eff. Report the raw base as well.
Missing weight	Not zero. Drop the row from weighted estimates, and investigate.

B8-L2 Confidence intervals and effect sizes

Wilson intervals with n_eff for weighted shares, and an effect size beside every p-value.

```
from statsmodels.stats.proportion import proportion_confint
low, high = proportion_confint(count=40, nobs=100, alpha=0.05, method="wilson")   #
  (0.309, 0.498)
```

Why not the Wald interval?	Zero width at p = 0 or 1, and too narrow for small n. Use Wilson.
Interval for a weighted share	Wilson with the weighted p and n_eff.
Significant vs meaningful	Large samples make trivial differences significant. Report the size with its interval.
Cohen's h	`2 * asin(sqrt(p1)) - 2 * asin(sqrt(p2))`. About 0.2 small, 0.5 medium, 0.8 large.

B8-L3 Many tests, and the false positives they bring

Define the family first, adjust within it with Holm or Benjamini-Hochberg, and report how many tests ran.

```
from statsmodels.stats.multitest import multipletests
reject, p_adj, _, _ = multipletests(p_values, alpha=0.05, method="holm")
```

Chance of at least one false positive in 20 tests at 5%	1 - 0.95^20 = 0.64.
Holm vs Bonferroni	Same family-wise control. Holm is step-down and rejects at least as many. Prefer Holm.
Benjamini-Hochberg	Controls the false discovery rate. Suits screening many cells.
Define the family	Before looking at results. Report how many tests ran.

B9 Experimental design

B9-L1 Power and sample size, before fieldwork

Fix baseline, MDE, alpha and power first, compute n, adjust for design effect and subgroups, then field.

```
z_a = NormalDist().inv_cdf(1 - alpha / 2)
z_b = NormalDist().inv_cdf(power)
p_bar = (p1 + p2) / 2
n = ((z_a * sqrt(2 * p_bar * (1 - p_bar)) + z_b * sqrt(p1 * (1 - p1) + p2 * (1 - p2))) / (p2 - p1)) ** 2
```

Four inputs to a sample size	Baseline, minimum detectable effect, alpha, power.
n per group, 40% vs 45%, alpha 0.05, power 0.8	1,534.
Halve the effect you want to detect	About four times the sample.
Weighted or subgroup reads	Multiply n by the design effect. Each subgroup needs the full n.

B9-L2 Analyse an A/B test the way it was randomised

Check the split, keep everyone in their assigned arm, analyse per randomised unit, and report the lift with its interval.

```
from scipy.stats import chisquare
chisquare([50_600, 49_400]).pvalue        # 0.000148, SciPy 1.17
```

```
from statsmodels.stats.proportion import confint_proportions_2indep
confint_proportions_2indep(1150, 10_000, 1000, 10_000)   # (0.0064, 0.0236)
```

First step in any A/B analysis	Sample ratio mismatch check: chi-square on arm counts against the plan.
Intention to treat	Analyse everyone in the arm they were assigned to, whatever happened next.
Unit of analysis	The unit of randomisation, usually the user, not the page view.
Report a lift as	The difference with its interval, compared with the planned minimum effect.

B9-L3 Peeking, novelty and other ways tests mislead

Plan the sample, duration and looks, analyse once at the horizon, and treat anything else as exploration.

Why peeking breaks a test	Each look is another chance for noise to cross $p < 0.05$. Ten looks gave 18.4% false positives in simulation.
Safe alternatives to daily peeking	Fixed horizon, or planned group-sequential looks with stricter thresholds.
Novelty effect	Lift that fades after launch. Run full weeks and compare week 1 with later weeks.
Interference	Treated units affect controls. Randomise at a level that contains the interaction.

B10 Feature engineering without leakage

B10-L1 Target leakage, features that know the answer

For every feature, know when it becomes available. Screen single-feature AUCs, and treat a near-perfect one as a suspect.

```
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y, X[col])
strength = max(auc, 1 - auc)
```

The one question for every feature	Would I know this value at prediction time?
Single-feature leakage screen	AUC of each feature alone. Flag max(auc, 1 - auc) near 1.
Common leaks	Consequences of the outcome, values measured after the prediction date, labelling proxies, order-encoding IDs.
Feature timeline	A table of when each feature becomes known, reviewed with the data owner.

B10-L2 Fit on training data only

Split first. Fit every learning step on training rows. Apply the fitted parameters to everything else.

```
params = fit_preprocessing(train)          # learn from train only
train_ready = apply_preprocessing(train, params)
test_ready = apply_preprocessing(test, params)   # reuse, never refit
```

Train/test contamination	A learning step saw test rows: imputation, scaling, encoding, selection.
Selection on all rows, noise data	CV accuracy 0.85. Inside the folds: 0.48. The first number is a leak.
The manual rule	Split first, fit on train, apply the fitted parameters to train and test.
Safe before the split	Steps that learn nothing from the rows: fixed recodes, unit conversions, parsing.

B10-L3 Time-safe features and point-in-time joins

Compute every feature as of the row's date, join with merge_asof, and split by time.

```
orders = orders.sort_values(["customer", "ordered_at"])
prior = orders.groupby("customer")["spend"].cumsum() - orders["spend"]
```

```
pd.merge_asof(preds.sort_values("as_of"), scores.sort_values("scored_at"),
              left_on="as_of", right_on="scored_at", by="customer",
              allow_exact_matches=False)
```

Prior-only running total	groupby(key)[col].cumsum() - col, after sorting by key and date.
Point-in-time join in pandas	pd.merge_asof(left, right, left_on, right_on, by=..., allow_exact_matches=False), both sorted.
merge_asof default for equal timestamps	allow_exact_matches=True: same-time rows match. Set False when they would not be known yet.
Split for forward-looking models	By time: train before a cut-off, test after. TimeSeriesSplit for CV.

B11 Machine learning workflow

B11-L1 Train, validation and test, split by the right unit

Tune on validation, measure once on test, and split by the unit that will be new in production.

```python
from sklearn.model_selection import GroupKFold, GroupShuffleSplit

splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=7)
train_idx, test_idx = next(splitter.split(df, groups=df["respondent_id"]))
```

Roles of train, validation and test	Fit, choose, measure once.
Rows repeat per respondent	Split by group (GroupKFold, GroupShuffleSplit), or the model learns people.
Group leakage demonstration	Random CV 0.995, GroupKFold 0.468 on a label unrelated to the features.
Rare class	stratify=y keeps the class share in each set.

B11-L2 Pipeline and ColumnTransformer

Put every learning step in one Pipeline, split columns with a ColumnTransformer, and deploy the fitted object itself.

```python
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
  NUMERIC),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), CATEGORICAL),
])
model = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
```

Why Pipeline	Every learning step refits per CV fold, and one fitted object goes to production.
OneHotEncoder default for unseen categories	handle_unknown="error": raises ValueError. Use "ignore" for production.
ColumnTransformer default for unlisted columns	remainder="drop": silently dropped.
Feature names after preprocessing	get_feature_names_out(), prefixed with step names such as num__age.

B11-L3 Cross-validation and honest hyperparameter search

Report CV mean and spread, tune with the reported metric, and estimate performance with nested CV or an untouched test set.

```python
res = cross_validate(pipe, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=7),
                     scoring=["roc_auc", "average_precision"])
res["test_roc_auc"].mean(), res["test_roc_auc"].std()
```

```python
search = GridSearchCV(pipe, grid, cv=inner, scoring="roc_auc")
honest = cross_val_score(search, X, y, cv=outer, scoring="roc_auc")
```

Report CV results as	Mean and spread across folds, for the metric you care about.
Pipeline parameter names in a grid	step__param, such as clf__C. make_pipeline names steps after their class.

Is best_score_ an honest estimate?	No. Noise simulation: 0.598 best_score_ against 0.515 nested. Use nested CV or a held-out test set.
Nested CV in scikit-learn	cross_val_score(GridSearchCV(pipe, grid, cv=inner), X, y, cv=outer).

B12 Model evaluation

B12-L1 Choose the metric from the decision

Choose metrics and thresholds from the decision. Report prevalence and counts beside every number.

Accuracy at 2.4% prevalence	0.976 for predicting all negatives. Useless as a headline.
Baseline for average precision	The prevalence. Report it beside the score.
Where the threshold comes from	The decision: review capacity or costs. Not the 0.5 default.
Precision and recall	Precision: share of flags that are real. Recall: share of real cases flagged.

B12-L2 Calibration, when the probability itself matters

When the probability is used as a number, check calibration, compare the Brier score with a constant baseline, and recalibrate on held-out data.

Calibrated means	Of cases scored p, about p are positive.
Check calibration	Calibration table or calibration_curve, plus Brier score against the constant prevalence baseline.
class_weight="balanced" and probabilities	Pushes them towards 0.5. Mean p went from 0.024 to 0.42 at 2.4% prevalence.
Recalibrate	CalibratedClassifierCV(model, method="sigmoid", cv=5). Isotonic only with plenty of data.

B12-L3 Imbalanced classes and error analysis by segment

Treat rare positives with a chosen threshold first, then break results down by segment and read the worst errors.

First response to rare positives	Train normally and choose the threshold from capacity or cost.
Resampling rule	Only inside training folds. Never the test set.
Error analysis	Metrics per segment with counts, then read the most confident errors.
Pooled vs segment recall	Pooling weights large segments. Overall 0.7 can hide a segment at 0.2.

B13 Interpretability and fairness

B13-L1 Permutation importance on held-out data

Measure importance by shuffling on held-out data, with the reported metric, and report the spread.

```sklearn
from sklearn.inspection import permutation_importance
result = permutation_importance(model, X_test, y_test, scoring="roc_auc",
                                n_repeats=10, random_state=0)
result.importances_mean, result.importances_std
```

Impurity importance bias	Favours high-cardinality features, from training data. A random ID scored 0.335 in this lesson's example.
Permutation importance	Shuffle a column on held-out data and measure the score drop, with its spread.
Correlated features	Share credit. Permute and report them as a group.
What importance means	What the model relies on. Not what causes the outcome.

B13-L2 SHAP values, and what they do not tell you

SHAP explains one prediction additively in a stated output space. It describes the model, not the world.

```
prediction = base_value + sum of the row's SHAP values
```

```shap
import shap
explainer = shap.TreeExplainer(model)
explanation = explainer(X_test)          # shap.Explanation
explanation.values, explanation.base_values
```

SHAP additivity	prediction = base_value + sum of the row's SHAP values, in the explainer's output space.
Output space for GradientBoostingClassifier in TreeExplainer	Log-odds (decision_function), not probability.
Global ranking from SHAP	Mean absolute SHAP value per feature.
What SHAP cannot show	Causes, a stable split between correlated features, behaviour outside the training range.

B13-L3 Performance by subgroup, and the fairness trade-offs

Measure rates per group with counts, choose the fairness property that fits the decision, and document the gaps.

Rates to report per group	n, positives, selection rate, TPR, FPR, precision, calibration.
Impossibility result	With different base rates, calibration and equal error rates cannot all hold (Kleinberg et al. 2016; Chouldechova 2017).
Dropping a protected attribute	Proxies keep its influence. Keep it for measurement where lawful.
Choosing a fairness property	From the harm: missed help (TPR), wrong flags (FPR), numbers used directly (calibration).

B14 Experiment tracking with MLflow

B14-L1 Track every run with MLflow

One run per fit, inside a with block, with parameters, commit, data hash, metrics and the model.

```mlflow
import mlflow

mlflow.set_experiment("churn")
with mlflow.start_run(run_name="lr-c1"):
```

```
    mlflow.log_params({"C": 1.0, "seed": 7, "features": "v3"})
    mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
    model = train(X_train, y_train, C=1.0, seed=7)
    mlflow.log_metric("cv_auc", cv_auc)
    mlflow.sklearn.log_model(model, name="model", input_example=X_train[:5])
```

What to log per run	Params (incl. seed), tags (git_commit, input_sha256), metrics, the model.
MLflow parameters	Stored as strings and immutable within a run.
Why with mlflow.start_run()	The run ends with FAILED status if training raises, instead of staying open.
log_model in MLflow 3	mlflow.sklearn.log_model(model, name="model"), with pinned requirements saved alongside.

B14-L2 Registry versions and aliases, not stages

Versions are fixed. Aliases move. Promote by comparison on the same data and metric, and keep the previous champion addressable.

```
from mlflow import MlflowClient

client = MlflowClient()
client.set_registered_model_alias("churn_model", "champion", "3")
model = mlflow.pyfunc.load_model("models:/churn_model@champion")
```

Version vs alias	A version is fixed. An alias is a movable label on a version.
Load the current production model	mlflow.pyfunc.load_model("models:/name@champion").
Model stages	Deprecated since MLflow 2.9, not supported in Unity Catalog. Use aliases.
Safe promotion	Compare on the same data and metric, require a margin, keep previous_champion for rollback.

B14-L3 Reproduce a run from its record

Code from the commit, environment from the lock, data checked by hash, settings converted back to types, metric compared within a tolerance.

```
run = mlflow.get_run(run_id)
run.data.params            # {'C': '1.0', 'seed': '7', 'features': 'v3'}  strings
run.data.tags["git_commit"], run.data.tags["input_sha256"]
run.data.metrics["cv_auc"]
reqs = mlflow.pyfunc.get_model_dependencies(f"runs:/{run_id}/model")   # path to
  requirements.txt
```

Reproduce a run	Commit, environment from lock, data verified by hash, typed params, rerun, compare within tolerance.
Parameters from mlflow.get_run	Strings. Convert with the expected types before use.
Model dependencies	mlflow.pyfunc.get_model_dependencies(model_uri) returns the path to requirements.txt.
Rerun differs beyond tolerance	Something that changes results was not logged. Find it and log it.